T11 Search Tools Decision Tree

Technical Intermediate

Choosing between Glob, Grep, Read, Agent and Bash based on context

The 5 native search tools

Claude Code offers 5 distinct tools to find and read information in a project. Each has a precise role, and mixing them without a strategy wastes tokens unnecessarily. `Glob` and `Grep` are in the default tool set on Windows only; on macOS, Linux and WSL, Claude searches with `find` and `grep` through `Bash` unless they are enabled (for example via `--tools`).

Tool	Role	Token cost
`Glob`	Find files by pattern	Low
`Grep`	Search content by regex	Low
`Read`	Read a full or partial file	High
`Agent`	Broad, multi-file exploration	Very high
`Bash`	System commands, uncovered cases	Variable

Glob: files by pattern

Glob returns a list of paths without reading their content. Use it to narrow the file set before reading content.

```
Pattern examples:
*.ts              → all .ts files at root
src/**/*.tsx      → all .tsx files in src/
**/*.test.ts      → all test files
```

Usage rule: use Glob first to narrow the scope, then Grep or Read on the resulting list.

Grep: content by regex

Grep uses ripgrep under the hood, making it suited to exact text searches. It reads files partially to extract only the matching lines, without loading the full content.

```
# Find a function by exact name
rg "authenticate" --type ts

# Search with context (3 lines around)
rg "validateJWT" -A 3
```

Use Grep for exact text searches. Do not escalate to Agent or Bash if the pattern is known.

Read: targeted reading

Read loads a full file or a line range via `offset` and `limit`. On large files, it automatically truncates at 2000 lines.

```
# Read lines 50 to 120 of a file
Read("auth.service.ts", offset=50, limit=71)
```

Always prefer a targeted Read with offset/limit over a full Read on a file with several hundred lines.

Agent: broad exploration

The Agent tool (formerly named Task) spawns a sub-agent with its own isolated context. It is the right option when you do not know where to look, the scope is wide, or exploration requires multiple files in parallel.

Cost: each sub-agent opens a new context, which represents a significant overhead. Reserve it for tasks that truly justify it.

Bash: the universal adapter

Bash covers cases the other 4 tools cannot handle directly: git commands, system calls, complex pipes, third-party CLI tools.

```
# Legitimate use cases for Bash
git log --oneline -20
find . -name "*.lock" -mtime -1
wc -l src/**/*.ts
```

Decision tree

```
Looking for files by name/extension
  → Glob

Looking for text/code and know the pattern
  → Grep

Want to read an identified file
  → Read (with offset/limit for long files)

Do not know where to look, broad scope
  → Agent (sub-agent)

None of the 4 tools is sufficient
  → Bash
```

Core principle: Glob and Grep before opening files with Read. Agent only when the scope is unknown.